

7. Find a procedure to launch virtual machine using trystack (Online Openstack Demo Version)

Aim:

To Find a procedure to launch virtual machine using trystack.

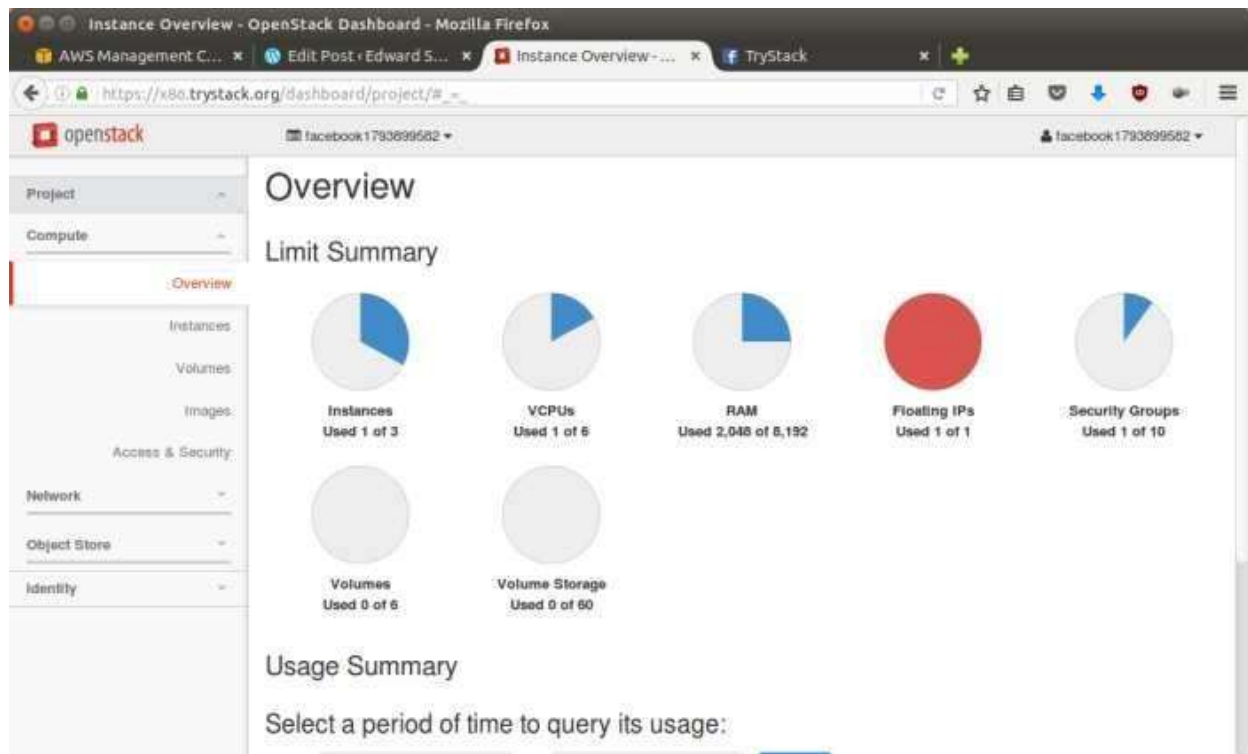
Steps:

OpenStack is an open-source software cloud computing platform. OpenStack is primarily used for deploying an infrastructure as a service (IaaS) solution like Amazon Web Service (AWS). In other words, you can *make your own AWS* by using OpenStack. If you want to try out OpenStack, **TryStack** is the easiest and free way to do it. In order to try OpenStack in TryStack, you must register yourself by joining **TryStack Facebook Group**. The acceptance of group needs a couple days because it's approved manually. After you have been accepted in the TryStack Group, you can log in TryStack.



TryStack.org Homepage

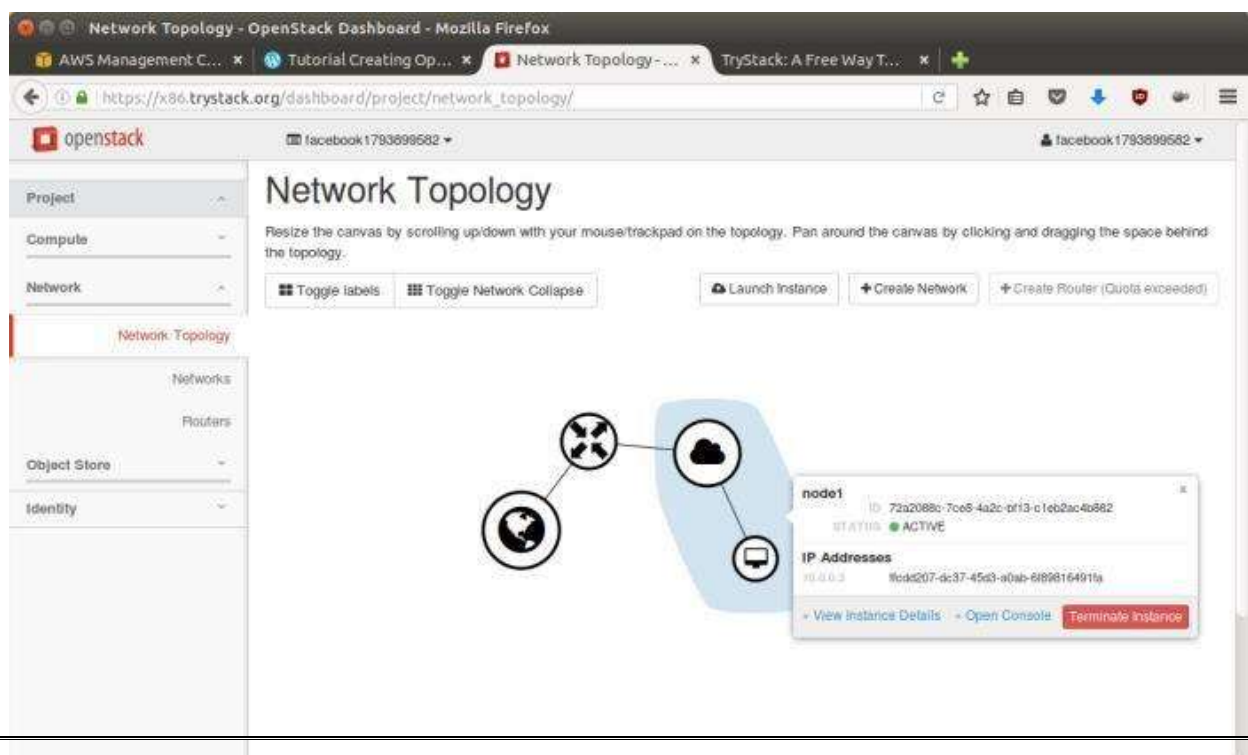
I assume that you already join to the Facebook Group and login to the dashboard. After you log in to the TryStack, you will see the Compute Dashboard like:



OpenStack Compute Dashboard

Overview: What we will do?

In this post, I will show you how to run an OpenStack instance. The instance will be accessible through the internet (have a public IP address). The final topology will like:



Network topology

As you see from the image above, the instance will be connected to a local network and the local network will be connected to internet.

Step 1: Create Network

Network? Yes, the network in here is our own local network. So, your instances will be not mixed up with the others. You can imagine this as your own LAN (Local Area Network) in the cloud.

1. Go to **Network > Networks** and then click **Create Network**.
2. In **Network** tab, fill **Network Name** for example `internal` and then click **Next**.
3. In **Subnet** tab,
 1. Fill **Network Address** with appropriate CIDR, for example `192.168.1.0/24`. Use **private network CIDR block** as the best practice.
 2. Select **IP Version** with appropriate IP version, in this case `IPv4`.
 3. Click **Next**.
4. In **Subnet Details** tab, fill **DNS Name Servers** with `8.8.8.8` (Google DNS) and then click **Create**.

Step 2: Create Instance

Now, we will create an instance. The instance is a virtual machine in the cloud, like AWS EC2. You need the instance to connect to the network that we just created in the previous step.

1. Go to **Compute > Instances** and then click **Launch Instance**.
2. In **Details** tab,
 1. Fill **Instance Name**, for example `Ubuntu 1`.
 2. Select **Flavor**, for example `m1.medium`.
 3. Fill **Instance Count** with `1`.
 4. Select **Instance Boot Source** with **Boot from Image**.
 5. Select **Image Name** with **Ubuntu 14.04 amd64 (243.7 MB)** if you want install Ubuntu 14.04 in your virtual machine.
3. In **Access & Security** tab,
 1. Click **[+]** button of **Key Pair** to import key pair. This key pair is a public and private key that we will use to connect to the instance from our machine.
 2. In **Import Key Pair** dialog,
 1. Fill **Key Pair Name** with your machine name (for example `Edward-Key`).
 2. Fill **Public Key** with your **SSH public key** (usually is in `~/.ssh/id_rsa.pub`). See description in Import Key Pair dialog box for more information. If you are using Windows, you can use **Puttygen** to generate key pair.
 3. Click **Import key pair**.
 3. In **Security Groups**, mark/check **default**.
4. In **Networking** tab,
 1. In **Selected Networks**, select network that have been created in Step 1, for example `internal`.

5. Click **Launch**.
6. If you want to create multiple instances, you can repeat step 1-5. I created one more instance with instance name Ubuntu 2.

Step 3: Create Router

I guess you already know what router is. In the step 1, we created our network, but it is isolated. It doesn't connect to the internet. To make our network has an internet connection, we need a router that running as the gateway to the internet.

1. Go to **Network > Routers** and then click **Create Router**.
2. Fill **Router Name** for example router1 and then click **Create router**.
3. Click on your **router name link**, for example router1, **Router Details** page.
4. Click **Set Gateway** button in upper right:
 1. Select **External networks** with **external**.
 2. Then **OK**.
5. Click **Add Interface** button.
 1. Select **Subnet** with the network that you have been created in Step 1.
 2. Click **Add interface**.
6. Go to **Network > Network Topology**. You will see the network topology. In the example, there are two network, i.e. external and internal, those are bridged by a router. There are instances those are joined to internal network.

Step 4: Configure Floating IP Address

Floating IP address is public IP address. It makes your instance is accessible from the internet. When you launch your instance, the instance will have a private network IP, but no public IP. In OpenStack, the public Ips is collected in a pool and managed by admin (in our case is TryStack). You need to request a public (floating) IP address to be assigned to your instance.

1. Go to **Compute > Instance**.
2. In one of your instances, click **More > Associate Floating IP**.
3. In **IP Address**, click Plus [+].
4. Select **Pool** to **external** and then click **Allocate IP**.
5. Click **Associate**.
6. Now you will get a public IP, e.g. 8.21.28.120, for your instance.

Step 5: Configure Access & Security

OpenStack has a feature like a firewall. It can whitelist/blacklist your in/out connection. It is called *Security Group*.

1. Go to **Compute > Access & Security** and then open **Security Groups** tab.
2. In **default** row, click **Manage Rules**.
3. Click **Add Rule**, choose **ALL ICMP** rule to enable ping into your instance, and then click **Add**.
4. Click **Add Rule**, choose **HTTP** rule to open HTTP port (port 80), and then click **Add**.
5. Click **Add Rule**, choose **SSH** rule to open SSH port (port 22), and then click **Add**.
6. You can open other ports by creating new rules.

Step 6: SSH to Your Instance

Now, you can SSH your instances to the floating IP address that you got in the step 4. If you are using Ubuntu image, the SSH user will be ubuntu.

Result:

Thus the openstack demo worked successfully.

8. Install Hadoop single node cluster and run simple applications like wordcount.

Aim:

To Install Hadoop single node cluster and run simple applications like wordcount.

Steps:

Install Hadoop

Step 1: [Click here](#) to download the Java 8 Package. Save this file in your home directory.

Step 2: Extract the Java Tar File.

Command: `tar -xvf jdk-8u101-linux-i586.tar.gz`

A terminal window titled 'edureka@localhost:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '[edureka@localhost ~]\$ tar -xvf jdk-8u101-linux-i586.tar.gz' is entered and executed, with a green cursor at the end of the line.

Fig: Hadoop Installation – Extracting Java Files

Step 3: Download the Hadoop 2.7.3 Package.

Command: `wget https://archive.apache.org/dist/hadoop/core/hadoop-2.7.3/hadoop-2.7.3.tar.gz`

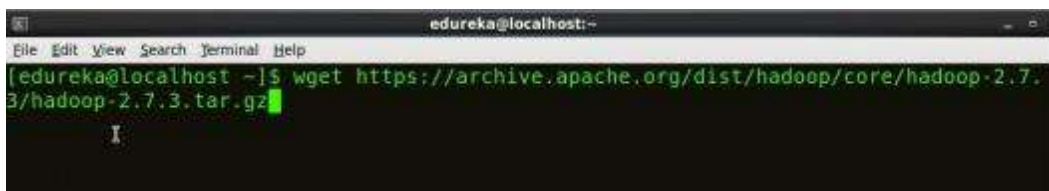
A terminal window titled 'edureka@localhost:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '[edureka@localhost ~]\$ wget https://archive.apache.org/dist/hadoop/core/hadoop-2.7.3/hadoop-2.7.3.tar.gz' is entered and executed, with a green cursor at the end of the line.

Fig: Hadoop Installation – Downloading Hadoop

Step 4: Extract the Hadoop tar File.

Command: `tar -xvf hadoop-2.7.3.tar.gz`



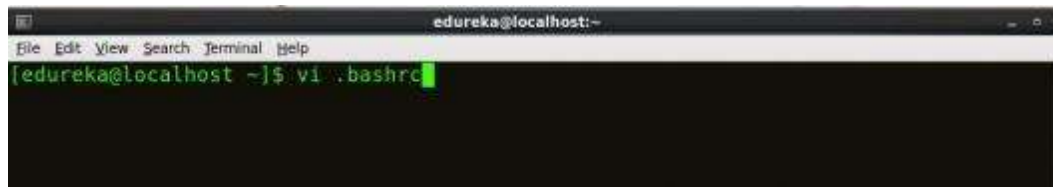
```
edureka@localhost:~ (on localhost.localdomain)
File Edit View Search Terminal Help
[edureka@localhost ~]$ tar -xvf hadoop-2.7.3.tar.gz
```

Fig: Hadoop Installation – Extracting Hadoop Files

Step 5: Add the Hadoop and Java paths in the bash file (.bashrc).

Open. **bashrc** file. Now, add Hadoop and Java Path as shown below.

Command: `vi .bashrc`



```
edureka@localhost:~
File Edit View Search Terminal Help
[edureka@localhost ~]$ vi .bashrc
```



```
# User specific aliases and functions

export HADOOP_HOME=$HOME/hadoop-2.7.3
export HADOOP_CONF_DIR=$HOME/hadoop-2.7.3/etc/hadoop
export HADOOP_MAPRED_HOME=$HOME/hadoop-2.7.3
export HADOOP_COMMON_HOME=$HOME/hadoop-2.7.3
export HADOOP_HDFS_HOME=$HOME/hadoop-2.7.3
export YARN_HOME=$HOME/hadoop-2.7.3
export PATH=$PATH:$HOME/hadoop-2.7.3/bin

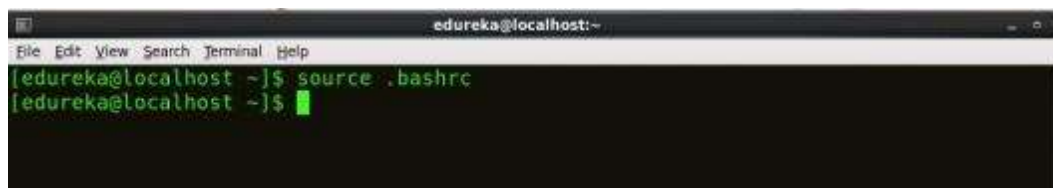
# Set JAVA_HOME
export JAVA_HOME=/home/edureka/jdk1.8.0_101
export PATH=/home/edureka/jdk1.8.0_101/bin:$PATH
```

Fig: Hadoop Installation – Setting Environment Variable

Then, save the bash file and close it.

For applying all these changes to the current Terminal, execute the source command.

Command: `source .bashrc`



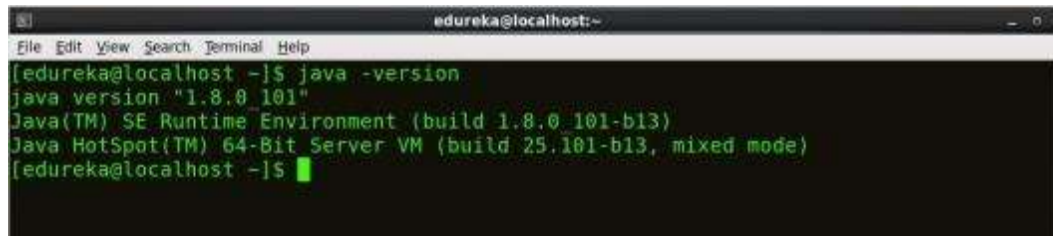
```
edureka@localhost:~
File Edit View Search Terminal Help
[edureka@localhost ~]$ source .bashrc
[edureka@localhost ~]$
```

Fig: Hadoop Installation – Refreshing environment variables

To make sure that Java and Hadoop have been properly installed on your system and can be accessed through the Terminal, execute the `java -version` and `hadoop`

version commands.

Command: java -version

A screenshot of a terminal window titled 'edureka@localhost:~'. The terminal has a menu bar with 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The command '[edureka@localhost ~]\$ java -version' has been entered and executed. The output is displayed in green text on a black background: 'java version "1.8.0_101"', 'Java(TM) SE Runtime Environment (build 1.8.0_101-b13)', and 'Java HotSpot(TM) 64-Bit Server VM (build 25.101-b13, mixed mode)'. The prompt '[edureka@localhost ~]\$' is shown again at the bottom with a green cursor.

```
[edureka@localhost ~]$ java -version
java version "1.8.0_101"
Java(TM) SE Runtime Environment (build 1.8.0_101-b13)
Java HotSpot(TM) 64-Bit Server VM (build 25.101-b13, mixed mode)
[edureka@localhost ~]$
```

Fig: Hadoop Installation – Checking Java Version

Command: `hadoop version`

```
edureka@localhost:~$ hadoop version
Hadoop 2.7.3
Subversion https://git-wip-us.apache.org/repos/asf/hadoop.git -r baa91f7c6bc9cb92be5982de4719c1c8af91ccff
Compiled by root on 2016-08-18T01:41Z
Compiled with protoc 2.5.8
From source with checksum 2e4ce5f957ea4db193bce3734ff29ff4
This command was run using /home/edureka/hadoop-2.7.3/share/hadoop/common/hadoop-common-2.7.3.jar
edureka@localhost:~$
```

Fig: Hadoop Installation – Checking Hadoop Version

Step 6: Edit the **Hadoop Configuration files**.

Command: `cd hadoop-2.7.3/etc/hadoop/`



Command: `ls`

All the Hadoop configuration files are located in **hadoop-2.7.3/etc/hadoop** directory as you can see in the snapshot below:


```
edureka@localhost:~/hadoop-2.7.3/etc/hadoop
File Edit View Search Terminal Help
[edureka@localhost ~]$ cd hadoop-2.7.3/etc/hadoop/
[edureka@localhost hadoop]$ ls
capacity-scheduler.xml      httpfs-env.sh              mapred-env.sh
configuration.xml           httpfs-log4j.properties    mapred-queues.xml.template
container-executor.cfg      httpfs-signature.secret    mapred-site.xml.template
core-site.xml               httpfs-site.xml            slaves
hadoop-env.cmd              kms-acls.xml               ssl-client.xml.example
hadoop-env.sh               kms-env.sh                 ssl-server.xml.example
hadoop-metrics2.properties kms-log4j.properties       yarn-env.cmd
hadoop-metrics.properties  kms-site.xml               yarn-env.sh
hadoop-policy.xml           log4j.properties           yarn-site.xml
hdfs-site.xml               mapred-env.cmd
```

Fig: Hadoop Installation – Hadoop Configuration Files

Step 7: Open *core-site.xml* and edit the property mentioned below inside configuration tag:

core-site.xml informs Hadoop daemon where NameNode runs in the cluster. It contains configuration settings of Hadoop core such as I/O settings that are common to HDFS & MapReduce.

Command: vi core-site.xml

```
edureka@localhost:~/hadoop-2.7.3/etc/hadoop
File Edit View Search Terminal Help
[edureka@localhost hadoop]$ vi core-site.xml
```

```
<configuration>
<property>
<name>fs.default.name</name>
<value>hdfs://localhost:9000</value>
</property>
</configuration>
```

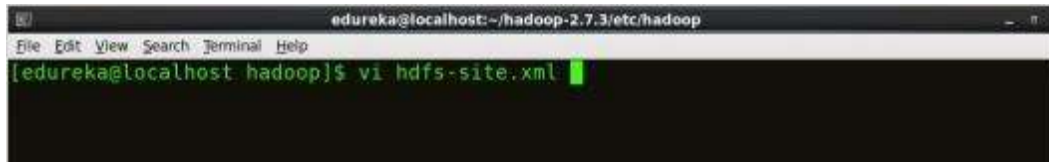
Fig: Hadoop Installation – Configuring core-site.xml

```
1      <?xml version="1.0" encoding="UTF-8"?>
2      <?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
3          <configuration>
4              <property>
5                  <name>fs.default.name</name>
6                  <value>hdfs://localhost:9000</value>
7              </property>
8          </configuration>
```

Step 8: Edit *hdfs-site.xml* and edit the property mentioned below inside configuration tag:

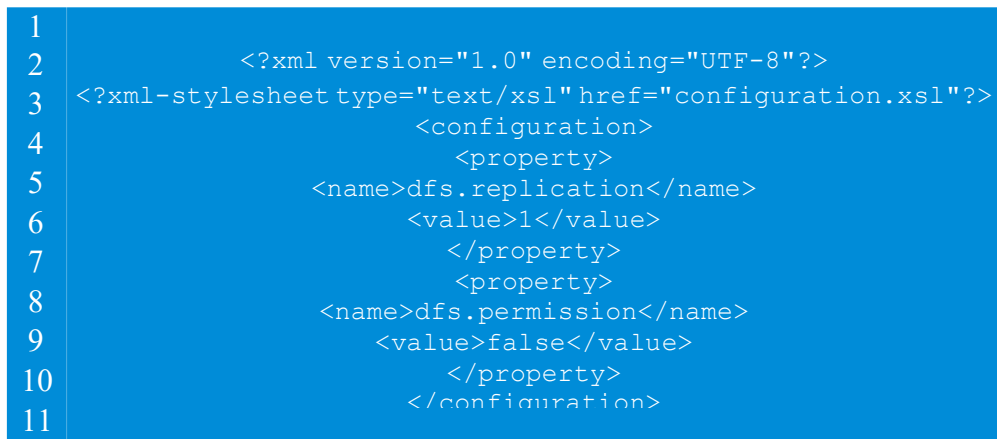
hdfs-site.xml contains configuration settings of HDFS daemons (i.e. NameNode, DataNode, Secondary NameNode). It also includes the replication factor and block size of HDFS.

Command: vi hdfs-site.xml

A terminal window with a dark background. The title bar shows 'edureka@localhost: ~/hadoop-2.7.3/etc/hadoop'. The command prompt is '[edureka@localhost hadoop]\$' and the command entered is 'vi hdfs-site.xml'. A green cursor is at the end of the command.A terminal window showing the XML content of hdfs-site.xml. The text is as follows:

```
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
<property>
<name>dfs.permission</name>
<value>>false</value>
</property>
```

Fig: Hadoop Installation – Configuring hdfs-site.xml

A code block with a blue background and white text. It contains the XML content of hdfs-site.xml, with line numbers 1 through 11 on the left side.

```
1
2      <?xml version="1.0" encoding="UTF-8"?>
3  <?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
4      <configuration>
5          <property>
6              <name>dfs.replication</name>
7              <value>1</value>
8          </property>
9          <property>
10             <name>dfs.permission</name>
11             <value>>false</value>
            </property>
        </configuration>
```

Step 9: Edit the *mapred-site.xml* file and edit the property mentioned below

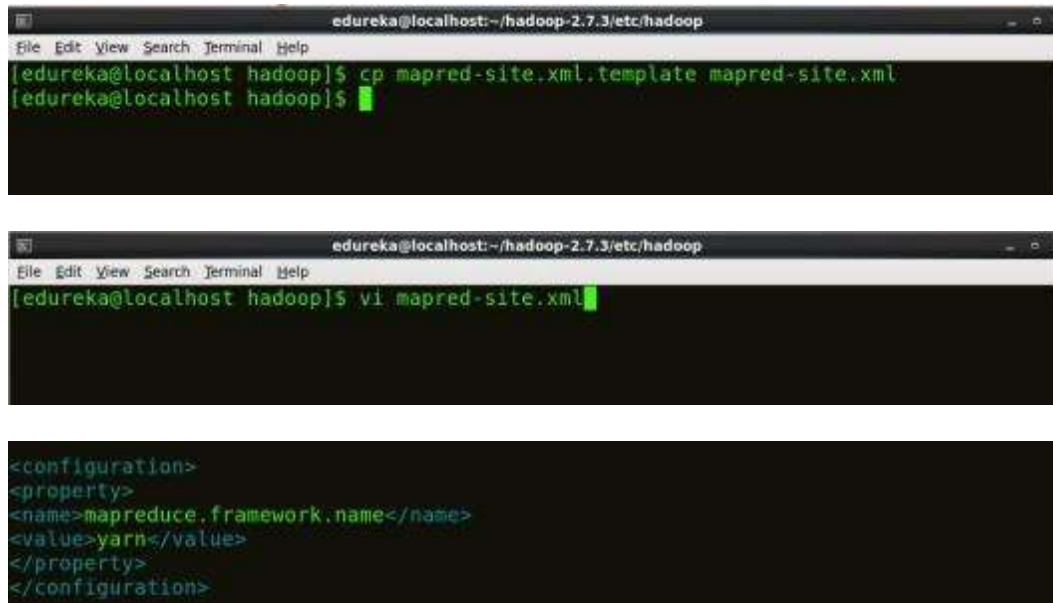
inside configuration tag:

mapred-site.xml contains configuration settings of MapReduce application like number of JVM that can run in parallel, the size of the mapper and the reducer process, CPU cores available for a process, etc.

In some cases, mapred-site.xml file is not available. So, we have to create the mapred-site.xml file using mapred-site.xml template.

Command: cp mapred-site.xml.template mapred-site.xml

Command: vi mapred-site.xml.



```
edureka@localhost:~/hadoop-2.7.3/etc/hadoop
File Edit View Search Terminal Help
[edureka@localhost hadoop]$ cp mapred-site.xml.template mapred-site.xml
[edureka@localhost hadoop]$ 
[edureka@localhost hadoop]$ vi mapred-site.xml
<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
</configuration>
```

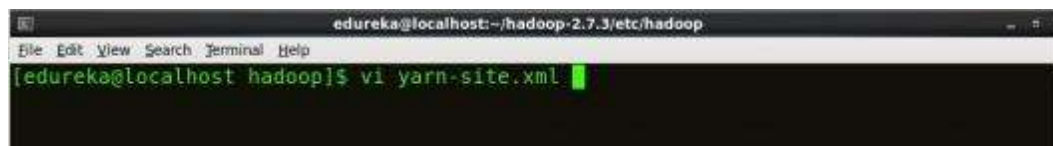
Fig: Hadoop Installation – Configuring mapred-site.xml

```
1      <?xml version="1.0" encoding="UTF-8"?>
2      <?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
3          <configuration>
4              <property>
5                  <name>mapreduce.framework.name</name>
6                  <value>yarn</value>
7              </property>
8          </configuration>
```

Step 10: Edit *yarn-site.xml* and edit the property mentioned below inside configuration tag:

yarn-site.xml contains configuration settings of ResourceManager and NodeManager like application memory management size, the operation needed on program & algorithm, etc.

Command: vi yarn-site.xml



```
edureka@localhost:~/hadoop-2.7.3/etc/hadoop
File Edit View Search Terminal Help
[edureka@localhost hadoop]$ vi yarn-site.xml
```

```

<configuration>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
<property>
<name>yarn.nodemanager.auxservices.mapreduce.shuffle.class</name>
<value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
</configuration>

```

Fig: Hadoop Installation – Configuring yarn-site.xml

```

1
2
3      <?xml version="1.0">
4      <configuration>
5          <property>
6              <name>yarn.nodemanager.aux-services</name>
7              <value>mapreduce_shuffle</value>
8          </property>
9          <property>
10             <name>yarn.nodemanager.auxservices.mapreduce.shuffle.class</
11             name>
12             <value>org.apache.hadoop.mapred.ShuffleHandler</value>
13         </property>
14     </configuration>

```

Step 11: Edit *hadoop-env.sh* and add the Java Path as mentioned below:

hadoop-env.sh contains the environment variables that are used in the script to run Hadoop like Java home path, etc.

Command: vi *hadoop-env.sh*

```

edureka@localhost:~/hadoop-2.7.3/etc/hadoop
File Edit View Search Terminal Help
[edureka@localhost: hadoop]$ vi hadoop-env.sh

# The java implementation to use.
export JAVA_HOME=/home/edureka/jdk1.8.0_101

```

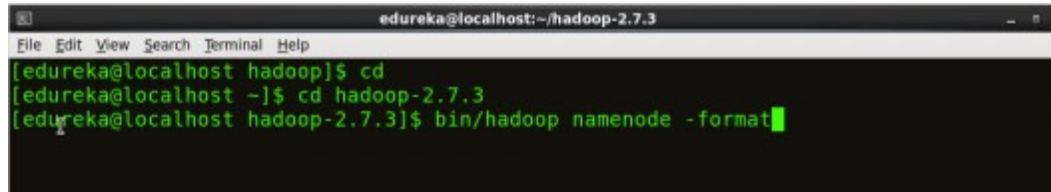
Fig: Hadoop Installation – Configuring hadoop-env.sh

Step 12: Go to Hadoop home directory and format the NameNode.

Command: cd

Command: cd hadoop-2.7.3

Command: bin/hadoop namenode -format

A terminal window titled 'edureka@localhost:~/hadoop-2.7.3' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal shows three lines of commands and their outputs: 1. '[edureka@localhost hadoop]\$ cd' followed by a new line. 2. '[edureka@localhost ~]\$ cd hadoop-2.7.3' followed by a new line. 3. '[edureka@localhost hadoop-2.7.3]\$ bin/hadoop namenode -format' followed by a green cursor. The text is green on a black background.

```
edureka@localhost:~/hadoop-2.7.3
File Edit View Search Terminal Help
[edureka@localhost hadoop]$ cd
[edureka@localhost ~]$ cd hadoop-2.7.3
[edureka@localhost hadoop-2.7.3]$ bin/hadoop namenode -format
```

Fig: Hadoop Installation – Formatting NameNode

This formats the HDFS via NameNode. This command is only executed for the first time. Formatting the file system means initializing the directory specified by the dfs.name.dir variable.

Never format, up and running Hadoop filesystem. You will lose all your data stored in the HDFS.

Step 13: Once the NameNode is formatted, go to hadoop-2.7.3/sbin directory and start all the daemons.

Command: cd hadoop-2.7.3/sbin

Either you can start all daemons with a single command or do it individually.

Command: ./start-all.sh

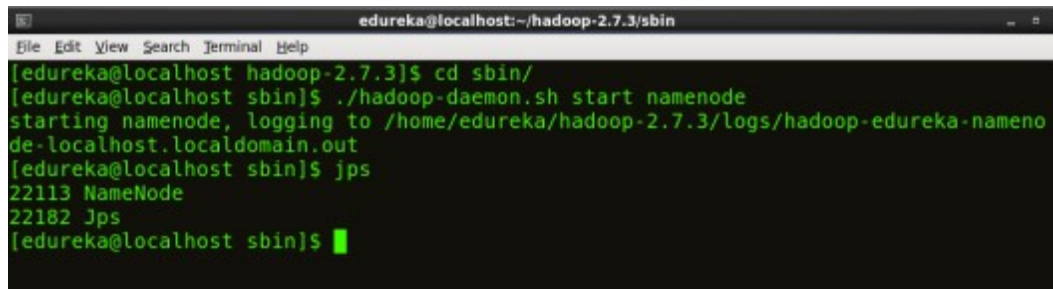
The above command is a combination of **start-dfs.sh**, **start-yarn.sh** & **mr-jobhistory-daemon.sh**

Or you can run all the services individually as below:

Start NameNode:

The NameNode is the centerpiece of an HDFS file system. It keeps the directory tree of all files stored in the HDFS and tracks all the file stored across the cluster.

Command: ./hadoop-daemon.sh start namenode

A terminal window titled 'edureka@localhost:~/hadoop-2.7.3/sbin' showing the execution of the command to start the NameNode. The user enters 'cd sbin/' and then './hadoop-daemon.sh start namenode'. The output shows the NameNode starting and logging to a specific file. Finally, the user enters 'jps' and the output shows '22113 NameNode' and '22182 Jps'.

```
edureka@localhost:~/hadoop-2.7.3/sbin
File Edit View Search Terminal Help
[edureka@localhost hadoop-2.7.3]$ cd sbin/
[edureka@localhost sbin]$ ./hadoop-daemon.sh start namenode
starting namenode, logging to /home/edureka/hadoop-2.7.3/logs/hadoop-edureka-namenode-localhost.localdomain.out
[edureka@localhost sbin]$ jps
22113 NameNode
22182 Jps
[edureka@localhost sbin]$
```

Fig: Hadoop Installation – Starting NameNode

Start DataNode:

On startup, a DataNode connects to the Namenode and it responds to the requests from the Namenode for different operations.

Command: ./hadoop-daemon.sh start datanode

A terminal window titled 'edureka@localhost:~/hadoop-2.7.3/sbin' showing the execution of the command to start the DataNode. The user enters './hadoop-daemon.sh start datanode'. The output shows the DataNode starting and logging to a specific file. Finally, the user enters 'jps' and the output shows '22113 NameNode', '22278 Jps', and '22206 DataNode'.

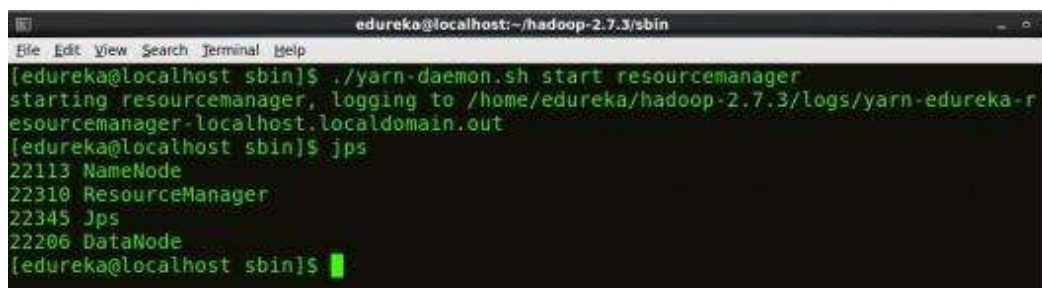
```
edureka@localhost:~/hadoop-2.7.3/sbin
File Edit View Search Terminal Help
[edureka@localhost sbin]$ ./hadoop-daemon.sh start datanode
starting datanode, logging to /home/edureka/hadoop-2.7.3/logs/hadoop-edureka-datanode-localhost.localdomain.out
[edureka@localhost sbin]$ jps
22113 NameNode
22278 Jps
22206 DataNode
[edureka@localhost sbin]$
```

Fig: Hadoop Installation – Starting DataNode

Start ResourceManager:

ResourceManager is the master that arbitrates all the available cluster resources and thus helps in managing the distributed applications running on the YARN system. Its work is to manage each NodeManagers and the each application's ApplicationMaster.

Command: ./yarn-daemon.sh start resourcemanager

A terminal window titled 'edureka@localhost:~/hadoop-2.7.3/sbin' showing the execution of the command to start the ResourceManager. The user enters './yarn-daemon.sh start resourcemanager'. The output shows the ResourceManager starting and logging to a specific file. Finally, the user enters 'jps' and the output shows '22113 NameNode', '22310 ResourceManager', '22345 Jps', and '22206 DataNode'.

```
edureka@localhost:~/hadoop-2.7.3/sbin
File Edit View Search Terminal Help
[edureka@localhost sbin]$ ./yarn-daemon.sh start resourcemanager
starting resourcemanager, logging to /home/edureka/hadoop-2.7.3/logs/yarn-edureka-resourcemanager-localhost.localdomain.out
[edureka@localhost sbin]$ jps
22113 NameNode
22310 ResourceManager
22345 Jps
22206 DataNode
[edureka@localhost sbin]$
```

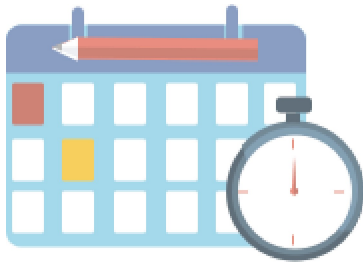

Fig: Hadoop Installation – Starting ResourceManager

Start NodeManager:

The NodeManager in each machine framework is the agent which is responsible for managing containers, monitoring their resource usage and reporting the same to the ResourceManager.

Command: `./yarn-daemon.sh start nodemanager`

```
edureka@localhost:~/hadoop-2.7.3/sbin
File Edit View Search Terminal Help
[edureka@localhost sbin]$ ./yarn-daemon.sh start nodemanager
starting nodemanager, logging to /home/edureka/hadoop-2.7.3/logs/yarn-edureka-nodemanager-localhost.localdomain.out
[edureka@localhost sbin]$ jps
22592 Jps
22113 NameNode
22310 ResourceManager
22206 DataNode
22559 NodeManager
[edureka@localhost sbin]$
```



[See Batch Details](#)

Fig: Hadoop Installation – Starting NodeManager

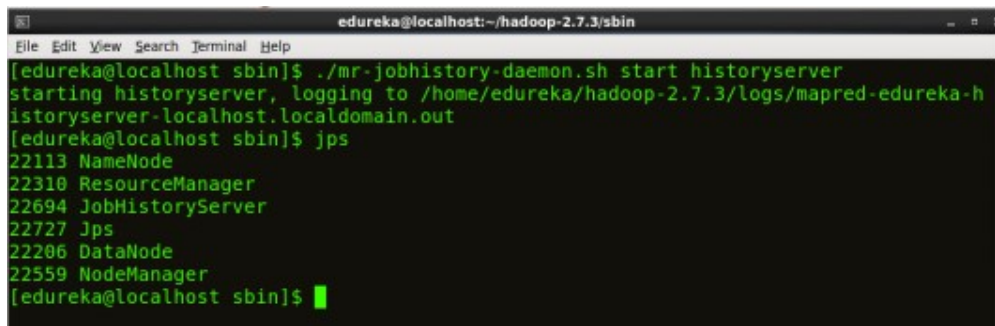
Start JobHistoryServer:

JobHistoryServer is responsible for servicing all job history related requests from client.

Command: `./mr-jobhistory-daemon.sh start historyserver`

Step 14: To check that all the Hadoop services are up and running, run the below command.

Command: jps



```
edureka@localhost:~/hadoop-2.7.3/sbin
[edureka@localhost sbin]$ ./mr-jobhistory-daemon.sh start historyserver
starting historyserver, logging to /home/edureka/hadoop-2.7.3/logs/mapred-edureka-h
istoryserver-localhost.localdomain.out
[edureka@localhost sbin]$ jps
22113 NameNode
22310 ResourceManager
22694 JobHistoryServer
22727 Jps
22206 DataNode
22559 NodeManager
[edureka@localhost sbin]$
```

Fig: Hadoop Installation – Checking Daemons

Step 15: Now open the Mozilla browser and go to **localhost:50070/dfshealth.html** to check the NameNode interface.

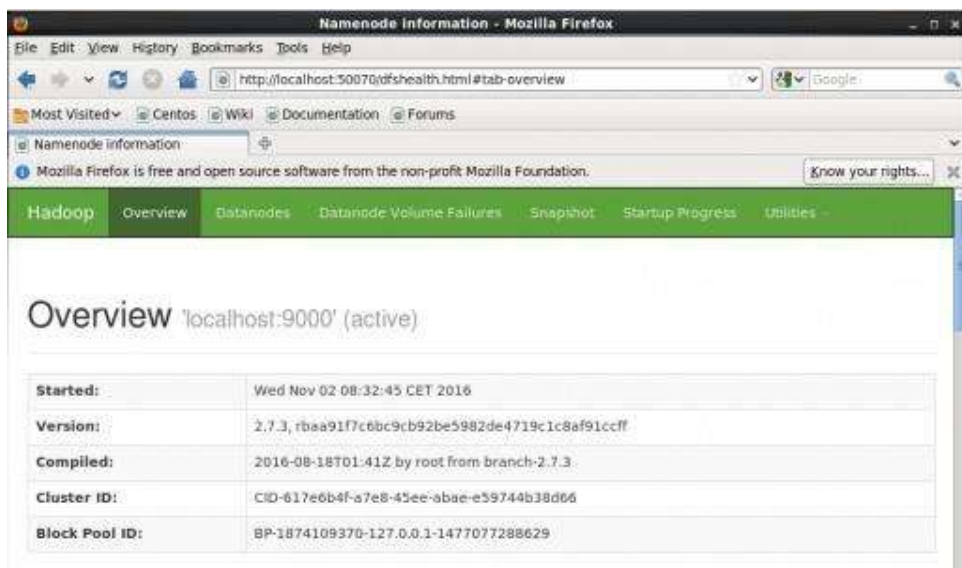


Fig: Hadoop Installation – Starting WebUI

Congratulations, you have successfully installed a single node Hadoop cluster

Result:

Thus the Hadoop one cluster was installed and simple applications executed successfully.